

# Mu Ruotong 225040014 Image Assignment1

225040014@link.cuhk.edu.cn

## Abstract

*This report details the implementation and evaluation of three core computer vision tasks. Section 1 covers the mathematical framework and execution of 2D affine transformations. Section 2 explores 3D-to-2D perspective projection using a pinhole camera model to visualize a geometric cube. Finally, Section 3 discusses the training of a U-Net architecture for portrait segmentation on the EG1800 dataset, including performance enhancements via data augmentation and advanced loss functions.*

## 1. Task 1: Geometric Affine Transformation

**Methodology** The objective is to apply a combined transformation—scaling ( $s$ ), rotation ( $\theta$ ), and translation ( $\mathbf{t}$ )—relative to the image center  $\mathbf{c}$ . For any given coordinate  $\mathbf{p}$ , the mapping function is defined as:

$$\mathbf{p}' = s \cdot \mathbf{R}(\theta) \cdot (\mathbf{p} - \mathbf{c}) + \mathbf{c} + \mathbf{t}, \quad (1)$$

where the rotation matrix  $\mathbf{R}(\theta)$  is given by:

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \quad (2)$$

In our implementation, we define three control points around the center and calculate their target positions using the above formulation. We then utilize `cv2.getAffineTransform` to derive the  $2 \times 3$  transformation matrix, followed by `cv2.warpAffine` with bilinear interpolation to generate the final output.

### Procedural Steps

1. Identify the image center  $\mathbf{c}$  and select a triplet of non-collinear reference points.
2. Map these points to their new locations by applying rotation, scaling, and translation sequentially.
3. Solve for the affine matrix using the source and target point pairs.
4. Perform the image warp, ensuring black padding for areas outside the original boundaries.

**Experimental Results** Fig. 1 illustrates the results using parameters:  $s = 0.5$ ,  $\theta = 45^\circ$ , and  $\mathbf{t} = (50, -50)$ . The output correctly reflects a 50% reduction in size, a  $45^\circ$  counter-clockwise rotation, and the specified pixel shifts. By centering the transformation at  $\mathbf{c}$ , we ensure that the rotation occurs around the object's centroid rather than the origin  $(0, 0)$ .



Figure 1. Comparison between the source image and the resulting affine transformation.

## 2. Task 2: 3D Wireframe Projection

**Mathematical Foundation** We employ a standard pinhole camera model to project 3D coordinates onto a 2D retina plane. The projection is governed by the intrinsic matrix  $\mathbf{K}$ :

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \quad (3)$$

For a 3D point in the camera coordinate system  $\mathbf{X}_c = (X, Y, Z)^\top$ , the image plane coordinates  $\mathbf{x}_s$  are obtained via:

$$\mathbf{x}_s = \begin{bmatrix} u/w \\ v/w \end{bmatrix}, \quad \text{where} \quad \begin{bmatrix} u \\ v \\ w \end{bmatrix} = \mathbf{K}\mathbf{X}_c. \quad (4)$$

The 3D cube is constructed by defining eight vertices, which are transformed according to the cube's translation and Euler angle rotations.

**Analysis of Projections** As shown in Fig. 2, the wireframe exhibits perspective distortion (vanishing point effect), which is a hallmark of the pinhole model. By altering the focal length  $(f_x, f_y)$ , we can simulate zooming effects.

Furthermore, increasing the  $Z$ -depth significantly reduces the projected footprint of the cube, confirming the inverse relationship between depth and perceived scale.

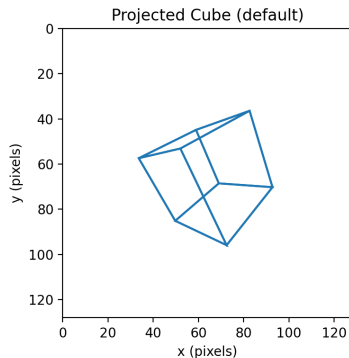


Figure 2. Perspective projection of a 3D cube at  $(0, 0, 2)$ .

### 3. Task 3: Portrait Segmentation Analysis

**Model Training** We trained a U-Net architecture for 20 epochs using the Adam optimizer with a learning rate of  $10^{-4}$  and a batch size of 8. The input resolution was set to  $224 \times 224$ . The training process tracked the evolution of loss and pixel-wise accuracy on the validation set. In the specific run shown in Fig. 3, the training loss decreased from 0.575 to 0.092, while the validation accuracy peaked at 0.9705 during epoch 19.

**Performance Evaluation** The learning curves in Fig. 3 demonstrate a stable convergence pattern. The consistent decline in both training and validation losses indicates that the model is learning effectively without significant overfitting. The validation accuracy plateaus towards the final epochs, suggesting that the model has reached its optimal capacity under the current hyperparameter configuration.

**Proposed Enhancements** To further boost performance, we integrated the following strategies:

- **Augmentation:** Applying random flips and color jittering to enhance generalization.
- **Loss Function:** Using a hybrid loss (Cross-Entropy + Dice Loss) to mitigate class imbalance issues common in segmentation.
- **Scheduling:** Implementing a cosine annealing learning rate to refine the weights in later stages.

**Face-aware Preprocessing & SAM2** By employing a Haar cascade detector, we implement a face-centric cropping strategy. This ensures the U-Net focuses on the region

of interest, significantly reducing background noise. Additionally, we compare our results with Segment Anything Model 2 (SAM2). While U-Net is optimized for this specific domain, SAM2 provides superior zero-shot boundaries but may require more precise prompting (boxes/points) for off-center subjects.

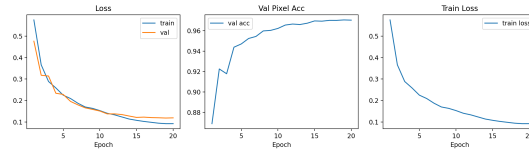


Figure 3. Training metrics: Loss and validation accuracy over 20 epochs.

### 4. Conclusion

This assignment successfully demonstrated the application of geometric transformations and 3D vision principles. Through Task 3, we observed how deep learning models like U-Net can be effectively trained and optimized for specialized tasks like portrait segmentation. Future work could involve exploring more complex backbones like Vision Transformers (ViT) to further enhance mask precision.

### References